## Agenda:

1. Hash Function + Collision + Chaining
2. Longest Substring Without Repeat
3. Length of longest consecutive sequence

---

## Hash Function + Collision + Chaining

Let's start with an example:

Given an array of size **N** and **Q** queries. In each query, an element is given.

We have to check whether that element exists or not in the given array.

A = [2, 4, 11, 15 , 6, 8, 14, 9]

**Queries:** K = 4 (return true) K = 10 (return false)

K = 17 (return false) K = 14 (return true)

**Bruteforce** → ∀ query, travel the array & check if K is present in array.

TC = $O(Q * N)$  SC = $O(1)$

**Sol 2** →

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| A = | 2 | 4 | 1 | 0 | 8 | 2 |

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| vst = | T | T | T | F | T | F | F | F | T | F |

vst[i] = true ⇒ i is present in A[]

```
∀i, vst[i] = false
for i → 0 to (N-1) {
    vst[A[i]] = true
}
```

| | 0 | 1 | 2 |
|---|---|---|---|
| A = | 2 | 83 | 10000 |

```
for i → 0 to (Q-1) {
    K = query[i]
    if (vst[K])  print ("True")
    else  print ("False")
}
```

TC = $O(N + Q)$  SC = $O(Range)$

1) High range → SC too much
2) Waste memory if data is scattered.

Limit the memory

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| A = | 2 ✓ | 53 ✓ | 81 ✓ | 102 ✓ | 93 | 7 | 18 |

M = 10

| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|
| | T | T | T | | | | | | |

$f(A[i]) \rightarrow [0 \quad 9] \leftarrow (A[i] \% 10)$

Hash Function

$2 \% 10 = 2$

$53 \% 10 = 3$

$81 \% 10 = 1$

$\boxed{102} \% 10 = 2$

collison $\rightarrow$ $\boxed{hf(x_1) = hf(x_2)}$

size of (A) $\rightarrow 10^6$ $\qquad$ $M = 10^5$

Pigeon Hole Principle $\rightarrow$ If there are N pigeons & (N-1) holes then there will be atleast one hole with more than one pigeon.

Impossible to have 1:1 mapping $\Rightarrow$ collison cannot be avoided.

But collison can be managed

Right hash function can minimise collision.

## Chaining

| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| 12 | 53 | 81 | 102 | 93 | 7 | 18 | 33 | 63 |

A = [12 53 81 102 93 7 18 33 63]

| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|
| | 81 | 12 | 53 | | | | 7 | 18 | |
| | | ↓ | ↓ | | | | | | |
| | | 102 | 93 | | | | | | |
| | | | ↓ | | | | | | |
| | | | 33 | | | | | | |
| | | | ↓ | | | | | | |
| | | | 63 | | | | | | |

Balanced BST → TC = $O(\log_2(N))$

A[i] % 10 →

| | | |
|---|---|---|
| 12 % 10 = 2 | 102 % 10 = 2 | 18 % 10 = 8 |
| 53 % 10 = 3 | 93 % 10 = 3 | 33 % 10 = 3 |
| 81 % 10 = 1 | 7 % 10 = 7 | 63 % 10 = 3 |

Worst case TC per query = $O(N)$

```java
// --- Input ---
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
int A[] = new int[n];
for(int i = 0; i < n; i++)  A[i] = sc.nextInt();

// --- Initialising array of List ---
int m = sc.nextInt(); // size of list
ArrayList<Integer> list[] = new ArrayList[m];
for(int i = 0; i < m; i++)  list[i] = new ArrayList<>();

// --- Chaining ---
for(int i = 0; i < n; i++) {
    int k = A[i] % m;    // Hash Function to limit range [0  m-1]
    list[k].add(A[i]);  // Add A[i] in k'th list
}
```

```java
int q = sc.nextInt();   // Count of queries
for(int i = 0; i < q; i++) {
    int x = sc.nextInt(); // Query Input
    int j = x % m;      // Map it to range [0  m-1]
    boolean found = false;
    for(int element: list[j]) {
        if(element == x) {
            found = true;
            break;
        }
    }
    if(found)   System.out.println("Found the element: " + x);
    else    System.out.println("Not found the element: " + x);
}

// --- Print List ---
for(int i = 0; i < m; i++) {
    System.out.print(i + "-> ");
    for(int e : list[i])    System.out.print(e +" ");
    System.out.println();
}
```

## Longest Substring Without Repeating Characters

**Q1:** Given a string s, find the **length** of the longest substring without repeating characters.

Example: Input: "**abcabcbb**" Output: **3**

Input: "**bbbbb**" Output: **1**

Bruteforce → ∀ substrings, check if it has unique characters, if yes → consider its length for answer.

$\frac{N*(N+1)}{2}$ $O(N)$

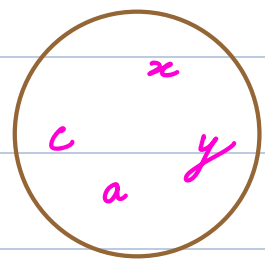
$TC = O(N^3)$

Sol

| | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| r | r | r | r | r | r | r | r | r | r | r | r |
| l | l | l | l | l | l | l | l | l | | | |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| a | b | e | c | d | a | d | x | c | y | x | a |

ans = ~~0~~ 5

check if char is present in substring → HashSet




```
// HashSet<characters> hs.
l = 0        r = 0        ans = 0
while (r < N) {
    ch = A[r]
    while (hs.contains(ch)) {
        c = A[l]
```

```
        hs.remove(c)        l++
    }

    hs.add(ch)

    ans = max(ans, r - l + 1)

    r++
}
```

TC = O(N)  SC = O(N)

## Longest Consecutive Sequence

**Q2:** You are given an array A[ ] (unsorted) of integers. Find the **length** of the longest set of consecutive integers. Consecutive means: x, x+1, x+2, x+3, … (order in the array does not matter)

Example: A = [100, 4, 200, 1, 3, 2]

The consecutive set: [1, 2, 3, 4] → length = **4**

A = [8, 4, 9, 5, 7, 2, 6, 0]

The consecutive set: [4, 5, 6, 7, 8, 9] → length = **6**

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| A = [ | 0 | 50 | 8 | 2 | 51 | 1 | 7 ] |

Ans = 3 {0, 1, 2}

Sol 1 → Sort the array & check for longest consecutive sequence.

TC = O(N log N)

Sol 2 →

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|---|
| A = [ | 99 | 8 | 10 | 2 | 11 | 9 | 4 | 3 | 7 ] |
| | ✓ | ✗ | ✗ | ✓ | ✗ | ✗ | ✗ | ✗ | ✓ |

∀ A[i], check if it is the start, if yes → find length of LCS.

checks

98 ✗ (8-1) = 7 ✓ 1 ✗ 10 ✓ 3 ✓ 6 ✗

100 ✗ (10-1) = 9 ✓ 3 ✓ 8 ✓ 2 ✓ 8 ✓ 9 ✓

4 ✓ 10 ✓

5 ✗ 11 ✓ 12 ✗

99 2 3 4

7 8 9 10 11

ans = ~~4~~ ~~3~~ 5

check (A[i] - 1)
check (A[i] + 1)

$TC = O(2N) \rightarrow \underline{O(N)}$

```
// ∀i, hs.add (A[i])
ans = 0
for i → 0 to (N-1) {
    if ( ! hs. contains (A[i] - 1)) {   // start
        len = 1      x = A[i]
        while ( hs. contains (x+1)) {
            len ++      x++
        }
        ans = max (ans, len)   // len
    }
}
return ans
```

$TC = \underline{O(N)}$ $SC = \underline{O(N)}$

**What will be the output for following code snippet?**

```java
public class Main {
    public static void main(String[] args) {
        int[] arr = {2, 4, 11, 15};
        int[] dat = new int[10];

        for (int x : arr) {
            dat[x] = 1;
        }

        int q1 = 11;
        System.out.println(dat[q1]);
    }
}
```

0 — 9

error

**What will be the output for following code snippet?**

```java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        int[] arr = {21, 31, 42};
        int[] table = new int[10];

        for (int x : arr) {
            table[x % 10] = x;    // collision overwrite
        }

        System.out.println(table[1]);
    }
}
```

21 % 10 = 1
31 % 10 = 1
42 % 10 = 2

1 → 31
2 → 42

→ 31